

BITE

Bestell- und Informationssystem für Takeaway-Essen

Das Projekt.

Der Trend, Essen online zu bestellen, ist ungebrochen. Deshalb entwickeln Sie eine App, die alle derzeit am Markt befindlichen Systeme (zumindest technologisch) alt aussehen lässt. Restaurants veröffentlichen ihre Speisekarte, hungrige Studierende bestellen ihre Lieblings Speisen.

Der Funktionsumfang in Kürze.

Der hier beschriebene Ablauf beschreibt die Anforderungen im Überblick. Die Punkte, die nur mit „API“ gekennzeichnet sind, sind ausschließlich im Backend als API umzusetzen. Für die Punkte, die als „Frontend“ markiert sind, ist auch eine Web-Oberfläche zu gestalten. Anforderungen, die mit dem Symbol  gekennzeichnet sind, müssen nur von Zweierteams ausgearbeitet werden. Alle anderen Anforderungen gelten für beide Gruppen.

	Funktion	API (SWK)	Frontend (WEA)
1	Ein Restaurant kann sich für BITE registrieren und seine wesentlichen Informationen (Name, Adresse, Öffnungszeiten, etc.) speichern.	✓	✓
2	Ein registriertes Restaurant kann sich mittels OAuth einloggen.		✓
3	Ein registriertes Restaurant kann seine Lieferbedingungen (Mindestbestellwert, Lieferkosten, etc.) speichern oder aktualisieren.	✓	
4	Ein registriertes Restaurant kann seine Speisekarte hochladen oder aktualisieren.	✓	✓
5	Studierende können Restaurants im Umkreis zu ihrer aktuellen Position suchen.	✓	✓
6	Studierende können die Speisekarte eines Restaurants abfragen.	✓	✓
7	Studierende können für eine Auswahl von Speisen den Preis inkl. Lieferkosten abfragen.	✓	✓
8	Studierende können eine Auswahl von Speisen bestellen.	✓	✓
9	Restaurants erhalten die Bestellung.	✓	
10	Restaurants können den Status einer Bestellung (in Zubereitung, in Zustellung etc.) verändern.	✓	✓

11	Restaurants können den Status einer Bestellung über einen Link ändern ().	✓	
12	Studierende können den aktuellen Status ihrer Bestellung abfragen.	✓	✓
13	Darstellung einer Bestellstatistik in Diagrammen.		✓

Nummer	1
Titel	Ein Restaurant kann sich registrieren.
Daten	• Name des Restaurants • Adresse inkl. GPS-Koordinaten • Öffnungszeiten (je Wochentag, etwaige Pausen berücksichtigen) • Webhook-URL für Bestellungen (siehe Anforderung 9) • Titelbild
Beispiele	Restaurant Nimmersatt Softwarepark 11, 4232 Hagenberg GPS: 48.368368883920425, 14.514449032326327 Montag Ruhetag Dienstag-Freitag: 11:00-15:00 und 17:00-22:00 Samstag/Sonntag: 11:00-14:00 Webhook-URL: https://api.nimmersatt.at/bite
Hinweise	• Die GPS-Koordinaten dürfen übergeben werden, es ist kein Geocoding der Adresse notwendig. • Bei einer erfolgreichen Registrierung soll die API einen API-Key liefern, der bei zukünftigen Requests des Restaurants als Authentifizierung genutzt werden muss.
Lernfelder	• Design des Datenmodells • Konzept zum Speichern von Bilddaten • Formulare, Eingabevalidierung und Services in Angular

Nummer	2
Titel	Ein registriertes Restaurant muss sich mittels OAuth einloggen können.
Daten	• Benutzername / Passwort
Hinweise	• Dies kann simuliert werden, eine tatsächliche Zuordnung zu den Restaurants im Backend muss nicht berücksichtigt werden. Im Wesentlichen geht es darum, dass zumindest der Front Channel abgesichert ist und nur registrierte Restaurants einen Zugang zu den abgesicherten Seiten/Daten haben. • Bei Einsatz eines Identity Servers, wie z.B. KeyCloak, können im Vorhinein Zugangsdaten für Restaurants im Backend hinterlegt werden. Versuchen Sie hier eine einfache Zuordnung zu simulieren (z. B. durch Einsatz von „Local Storage“). • Der Testserver aus der Übung kann verwendet werden (bedeutet gleicher Login für alle Restaurants), für eine Benotung mit „Sehr Gut“ sollte aber ein anderer Server konfiguriert werden.
Lernfelder	• OAuth & Open ID Connect

Nummer	3
Titel	Ein Restaurant kann seine Lieferbedingungen festlegen.
Daten	• Mindestbestellwert abhängig von der Entfernung • Lieferkosten abhängig von der Entfernung
Hinweise	• Ein Aufruf darf nur mit gültigem API-Key erfolgen.
Beispiele	• Unter 10 Kilometer: Mindestbestellwert 20 € Keine Lieferkosten • Ab 10 Kilometer: Mindestbestellwert 20 € Bestellwert bis 30 €: Lieferkosten: 5 € Bestellwert über 30 €: Keine Lieferkosten
Lernfelder	• Design des Datenmodells • Authentifizierte Requests • Objektorientiertes Design

Nummer	4
Titel	Ein Restaurant kann seine Speisekarte festlegen.
Daten	• Speisen und Getränke mit Namen, Beschreibung und Preis, gruppiert in Kategorien
Hinweise	• Es soll möglich sein, die gesamte Speisekarte mit einem Request zu aktualisieren (bspw. ein Formular) • Ein Aufruf darf nur mit gültigem API-Key erfolgen.
Beispiele	• Kategorie Pizza ○ Margherita Tomaten, Käse 9,59 € ○ Al Tonno Tomaten, Käse, Thunfisch, Zwiebel, Oliven 11 € ○ Spinaci Tomaten, Käse, Spinat, Feta 10 € • Kategorie Pasta ○ Lasagne al Forno mit Rinderfaschiertem 12 € ○ Spaghetti Frutti di Mare mit Meeresfrüchten in Weißweinsauce 14 €

Lernfelder	• API-Design • Design des Datenmodells • Authentifizierte Requests • Formulare, Eingabevalidierung und Services in Angular
Nummer	5
Titel	Studierende können Restaurants suchen.
Daten	• GPS-Koordinaten • Alle/nur derzeit geöffnete Restaurants anzeigen
Hinweise	• Die Antwort soll eine übersichtliche und optisch ansprechende Darstellung der Suchergebnisse ermöglichen • Die Restaurants sollen nach ihrer Entfernung zur suchenden Person sortiert sein. • Achten Sie auf eine gute Darstellung im Frontend (z.B. mittels Einsatz von Kartenkomponenten).
Beispiele	GPS: 48.368368883920425, 14.514449032326327 Nur derzeit geöffnet: ja Anzahl Restaurants: 10
Lernfelder	• API-Design • Distanzberechnung • Formulare, Services, Einsatz von Drittanbieter-Komponenten
Nummer	6
Titel	Studierende können die Speisekarte eines Restaurants abfragen.
Daten	• Eindeutige Identifikation des Restaurants
Hinweise	• Die gesamte Speisekarte des Restaurants soll als Antwort geliefert werden.
Lernfelder	• API-Design • Datenbankabfrage • Formulare, Services, Pipes und Routing

Nummer	7
Titel	Studierende können für eine Auswahl von Speisen den Preis inkl. Lieferkosten abfragen.
Daten	• Eindeutige Identifikation des Restaurants • Ein oder mehrere Produkte mit der Anzahl • Lieferadresse inkl. GPS-Koordinaten und Freitext
Hinweise	• Dieser Schritt dient zur Validierung einer Bestellung und zur Ermittlung des finalen Preises, um vor der Bestellung noch einmal eine Zusammenfassung anzeigen zu können. • Es ist zu berücksichtigen, dass eine Zustellung in der gewünschten Form aus verschiedenen Gründen nicht möglich ist (z. B. außerhalb des Liefergebiets, Mindestbestellmenge nicht erreicht, ungültige Speisenauswahl, ...).
Beispiele	• Restaurant: Nimmersatt • Bestellung: ◦ 1x Pizza Margherita ◦ 2x Lasagne Al Forno • Lieferadresse: - Studentenheim Hagenberg - Softwarepark 23, 4232 Hagenberg - GPS: 48.3692673, 14.5125979 • Freitext: Zimmer 404
Lernfelder	• API-Design • Datenbankabfrage • Lieferkosten-Berechnung • Formulare und Services und eventuell eine eigene Komponente für die Suche

Nummer	8
Titel	Studierende können eine Auswahl von Speisen bestellen.
Daten	• Eindeutige Identifikation des Restaurants • Ein oder mehrere Produkte mit der Anzahl • Lieferadresse inkl. GPS-Koordinaten und Freitext
Hinweise	• Mit diesem Schritt erfolgt eine verbindliche Bestellung. • Bei erfolgreicher Bestellung soll die API eine eindeutige Identifikation liefern, mit der der Status abgefragt werden kann (siehe Anforderung 10). • Außerdem wird eine Übermittlung der Bestellung an das jeweilige Restaurant ausgelöst (siehe Anforderung 9).
Beispiele	• Restaurant: Nimmersatt • Bestellung:

	○ 1x Pizza Margherita ○ 2x Lasagne Al Forno • Lieferadresse: Studentenheim Hagenberg Softwarepark 23, 4232 Hagenberg GPS: 48.3692673, 14.5125979 Zimmer 404
Lernfelder	• Speichern der Bestellung • Formulare und Services

Nummer	9
Titel	Restaurants erhalten die Bestellung.
Daten	• Eindeutige Identifikation der Bestellung • Ein oder mehrere Produkte mit der Anzahl • Lieferadresse inkl. GPS-Koordinaten und Freitext
Hinweise	• Restaurants stellen eine URL zur Verfügung (siehe 1), an die Sie eingehende Bestellungen schicken können. • Sie müssen diese APIs der Restaurants nicht selbst implementieren, Sie können auch Tools wie https://webhook.site oder https://request-bin.com verwenden. • Es kann passieren, dass die API eines Restaurants kurzfristig nicht erreichbar ist.
Beispiele	• Identifikation: 4711 • Bestellung: ○ 1x Pizza Margherita ○ 2x Lasagne Al Forno • Lieferadresse: Studentenheim Hagenberg Softwarepark 23, 4232 Hagenberg GPS: 48.3692673, 14.5125979,17.5 Zimmer 404
Lernfelder	• Gesichertes Aufrufen von externen APIs • Fehlerbehandlung • Ausführen von Aktionen im Hintergrund

Nummer	10
Titel	Restaurants können den Status einer Bestellung (in Zubereitung, in Zustellung, etc.) verändern.
Daten	• Eindeutige Identifikation der Bestellung • Status: In Zubereitung, in Zustellung, ...

Hinweise	• Ein Aufruf darf nur mit gültigem API-Key erfolgen.
Lernfelder	• API-Design • Authentifizierte Requests • Formulare und Services

Nummer	11
Titel	Restaurants können den Status einer Bestellung über Link ändern (👤)
Daten	• Eindeutige Identifikation der Bestellung mittels Token in Link • Neuer Status ist in generiertem Token enthalten
Hinweise	• Bestellung (siehe Anforderung 9) wird um Links erweitert, mit denen Statusänderungen durchgeführt werden können. Der Link enthält die zur Statusänderung notwendigen Daten. • Ein Aufruf darf nur mit gültigem API-Key erfolgen.
Lernfelder	• Generierung und Verwalten von Tokens • Authentifizierte Requests

Nummer	12
Titel	Studierende können den aktuellen Status ihrer Bestellung abfragen.
Daten	• Eindeutige Identifikation der Bestellung
Beispiele (Ergebnis)	• Status: An Restaurant übermittelt
Lernfelder	• Design des Datenmodells • Formulare und Services

Nummer	13 (nur für WEA)
Titel	Darstellung einer Bestellstatistik in Diagrammen. Für "Sehr Gut" in WEA notwendig.
Daten	• Zeitraum der ausgewertet werden soll. • Denken Sie sich sinnvolle Metriken aus. • Erweitern Sie dafür ihr Backend, sodass Sie diese Daten laden können. Daten, die für eine Statistik sinnvoll sind, aber nicht tatsächlich berechnet werden können, dürfen Sie simulieren.
Hinweise	• Dieser Aufruf darf nur authentifiziert erfolgen. • Der Zeitraum muss konfigurierbar sein.
Beispiele (Ergebnis)	• aktuelle Anzahl der Bestellungen pro Status pro Restaurant, nicht gelieferte / nicht abgeholte Bestellungen, etc.
Lernfelder	• Einsatz von zusätzlichen Komponenten für grafische Diagramme.

Technologische Hinweise

Setzen Sie die beschriebenen Anforderungen mit den aus SWK und WEA bekannten Technologien und Konzepten um. Implementieren Sie die Datenzugriffsschicht auf Basis von ADO.NET. Der Einsatz eines OR-Mapping-Werkzeugs (NHibernate, Entity Framework etc.) ist nicht gestattet, weil diese Technologien erst in einem späteren Semester ausführlich behandelt werden und sie nicht Bestandteil des zu bewertenden Stoffumfangs sind.

Die Entwicklung der Projektarbeit für SWK ist auf GitHub durchzuführen. Alle Teams bekommen ein Repository zugewiesen, auf welches das Team und die Übungsleiter Zugriff haben. Für den WEA-Teil darf das Repository weiterverwendet werden, die Entwicklung unter GitHub ist aber optional. Die finale Version der beiden Ausbaustufen in SWK sowie das Endergebnis des WEA-Teils ist auf die E-Learning-Plattform hochzuladen. Bei Zweierteams müssen beide Teammitglieder den Upload durchführen.

Hinweis: GitHub Classroom stellt nur sehr eingeschränkt Ressourcen für GitHub Actions zur Verfügung, die sich alle Mitglieder eines Classrooms, also alle Studierenden, teilen müssen. Setzen Sie daher ein Werkzeug ein, mit dem Sie GitHub-Workflows auch lokal ausführen können.

Dokumentation

Dokumentation SWK

Als Dokumentation zum Projekt bitten wir Sie um die Beantwortung der folgenden Fragen in Form einer README-Markdown-Datei im Stammverzeichnis Ihres Sourcecode-Repositories. Da wir – abgesehen von der Kommentierung des Quelltexts – keine weitere Dokumentation einfordern, erwarten wir, dass Sie sich mit diesen Fragen intensiv auseinandersetzen und diese ausführlich beantworten.

1. Für welches Datenmodell haben Sie sich entschieden? *ER-Diagramm, etwaige Besonderheiten erklären: Welche Entscheidungen mussten Sie treffen, wofür (und wogegen) haben Sie sich entschieden und warum?*
2. Dokumentieren Sie auf Request-Ebene den gesamten Workflow anhand eines durchgängigen Beispiels (von der Registrierung eines Restaurants bis zur Abfrage des Bestellstatus). Sie können ein Tool Ihrer Wahl einsetzen, z. B. Postman Workflows, VS Code, etc. *HTTP-Requests inkl. HTTP-Verb, URL, Parametern, Body und Headern*

3. Wie stellen Sie sicher, dass manche Requests nur mit einem gültigen API-Key aufgerufen werden können?
4. Bei welchen Teilen Ihres Systems ist eine korrekte Funktionsweise aus Sicht von BITE am wichtigsten? Welche Maßnahmen haben Sie getroffen, um sie zu gewährleisten?
5. Wie stellen Sie sicher, dass ihre API bei der Berechnung des Gesamtpreises für eine Bestellung inkl. Lieferkosten in allen Fällen ein korrektes Ergebnis liefert?
6. Welche Maßnahmen haben Sie getroffen, um sicherzustellen, dass alle Bestellungen tatsächlich beim Restaurant ankommen, auch wenn dessen API kurzfristig nicht erreichbar ist?
7. Ein Restaurant behauptet, keine Bestellungen übermittelt zu bekommen. Wie analysieren Sie das Problem?
8. Wie haben Sie sichergestellt, dass die Lieferbedingungen (Mindestpreis und Versandbedingungen) möglichst einfach erweiterbar sind? Beispielsweise könnten die Versandkosten von der Postleitzahl abhängig werden. *Beschreiben Sie die relevanten Stellen des Designs (Klassen-Diagramm).*
9. Denken Sie an die Skalierbarkeit Ihres Projekts: Plötzlich verwenden tausende Restaurants und zehntausende Studierende das Produkt. Was macht Ihnen am meisten Kopfzerbrechen?
10. Haben Sie im Zuge der Projektarbeit Erfahrungen mit dem Einsatz von KI-Agenten zur Generierung von Sourcecode gesammelt? Welche? *Was hat gut funktioniert, wo sind Sie gescheitert?*
11. Wenn Sie das Projekt neu anfangen würden – was würden Sie anders machen?

Dokumentation WEA

Die Dokumentation des Frontends (WEA) muss eine PDF-Datei sein, welche folgende Kernbestandteile enthält:

1. Inhaltsverzeichnis
2. Dokumentation ihrer Architektur: Beschreiben Sie die Architektur Ihres Projekts, so dass sich jemand Externer schnell in ihrem Projekt zurechtfindet, ohne lange den Quelltext durchforsten zu müssen (Strukturierung von Komponenten, Zusammenhang Services mit Komponenten, wichtige Komponenten). Setzen Sie hierfür z.B. Compodoc ein: <https://compodoc.app/>
3. Stellen Sie mögliche Navigationswege in Ihrer Anwendung grafisch dar.
4. Kompletter bebildeter Testlauf Ihrer Anwendung. Für jedes Szenario muss zumindest ein Screenshot vorhanden sein, der die Funktionalität zeigt. Fügen Sie auch notwendige Erläuterungen hinzu.
5. Beantworten Sie folgende Fragen:
 - a. Was ist zu tun, wenn sich URLs ändern? Wie invasiv ist der Eingriff in Ihre Anwendung, um diese zu ändern?
 - b. Wie stellen Sie sicher, dass bestimmte Seiten nur nach einem Login zugreifbar sind?
 - c. Wie stellen Sie eine korrekte Dateneingabe sicher?
 - d. Was passiert, wenn Aufrufe an das Backend Fehler produzieren?
6. Vor der Präsentation der Projektarbeit wird es einen Fragebogen in Moodle geben, Sie werden zeitgerecht per E-Mail informiert. Beantworten Sie diesen, ein Versäumnis wird mit einem schlechteren Notengrad bewertet.

Ausbaustufe 1 (SWK)

In der ersten Ausbaustufe sind die folgenden Aufgaben zu erledigen:

Definieren Sie Ihren Entwicklungsprozess

- ☐ Sourcecodeverwaltung einrichten
- ☐ Ordnerstruktur überlegen und umsetzen
- ☐ Zusammenarbeit in 2er-Teams (Branching, Pull Requests, etc.)
- ☐ Issue-Tracking-System einrichten (👤)
- ☐ Build-Pipeline einrichten (👤)

Erstellen Sie die grundlegende Projektstruktur

- ☐ Visual Studio-Solution mit Projektstruktur anlegen
- ☐ Vorbereitung der Dokumentation (README.md)
- ☐ Projekt(e) für automatisierte Tests anlegen

Implementierung (Hinweis: Aus vergangenen Jahren wissen wir, dass manche Entscheidungen in der Datenzugriffsschicht erst in der zweiten Ausbaustufe sinnvoll getroffen werden können. Arbeiten Sie also nicht zu weit vor und konzentrieren Sie sich vorerst auf die genannten Entitäten).

- ☐ Erstellen einer Testdatenbank (zumindest drei „liebervoll“ gepflegte Restaurants mit Speisekarten)
- ☐ Datenbank, zumindest mit den Entitäten für (1), (3) und (4)

- ☐ Implementierung der Datenzugriffsschicht für die bestehenden Entitäten
- ☐ Test der Datenzugriffsschicht (exemplarisch für zumindest eine Entität)

Ausbaustufe 2 (SWK)

In der zweiten Ausbaustufe sind die folgenden Aufgaben zu erledigen:

Implementierung der geforderten Funktionalität

- ☐ Erweiterung der Testdatenbank für die restlichen Entitäten (zumindest 20 Studierende (Kund*innen) und 30 Bestellungen in verschiedenen Zuständen).
- ☐ Design der API-Endpunkte
- ☐ Implementierung der API und Erweiterung der Datenzugriffsschicht
- ☐ Testen Sie automatisiert vor allem die Bereiche
 - die komplexere Logik beinhalten,
 - wo manuelles Testen aufwändig wäre oder
 - wo Sie zahlreiche Sonderfälle behandeln und testen müssen.
- ☐ Code-Dokumentation (wo sinnvoll)

Beantwortung der Konzeptfragen

- ☐ Finale Beantwortung sämtlicher Fragen

Ausfüllen des Feedbackfragebogens

Ausbaustufe 3 (WEA)

In der dritten Ausbaustufe sind die folgenden Aufgaben zu erledigen:

Implementierung der geforderten Funktionalität

- ☐ Ansprechendes Design des Web Frontends (hier geht es weniger um das Aussehen selbst, sondern um die übersichtliche Darstellung, Darstellung nur sinnvoller Daten, sowie logischer, intuitiver und einfacher Abläufe für Benutzer*innen; "Benutzbarkeit") Achtung: Die Benutzer*innen sind nicht die Entwickler*innen selber!
- ☐ Webanwendung mit Angular (Version >= 21), welche mit dem Backend über REST-Service-Schnittstellen kommuniziert. Die Services zum Backend dürfen NICHT generiert werden.
- ☐ Auswahl und Einsatz geeigneter CSS Frameworks, bzw. Angular Komponenten Bibliotheken (Bootstrap, Fomantic, Angular Material, DevExtreme, etc.).
- ☐ Auswahl und Einsatz geeigneter Services für OAuth-Login (KeyCloak, IdentityServer etc.).
- ☐ (Sinnvoller) Einsatz von zusätzlichen Angular-Konzepten wie Pipes, Direktiven, Modulen, etc. (siehe z.B. Angular Dokumentation), die nicht in der Übung verwendet wurden.
- ☐ Code-Dokumentation (wo sinnvoll)
- ☐ Dokumentation wie oben in der WEA-Sektion beschrieben
- ☐ Ausfüllen des Feedbackfragebogens

Hinweise für BB-Studierende

In der berufsbegleitenden Studienform von SE.ba findet die Lehrveranstaltung WEA (Web-Architekturen und Frameworks) erst im 5. Semester statt und daher ist dieser Teil erst bis zum Ende des 5. Semester abzugeben (der genau Abgabetermin wird im 5. Semester bekanntgegeben werden).

Auch wenn der WEA-Teil für Sie bei der Umsetzung erst später relevant wird, so sollten Sie das gesamte Design Ihrer SWK-WEA-Anwendung u.a. vom User Interface her betrachten! Damit kann nämlich die geforderte Funktionalität und Benutzerfreundlichkeit leichter gewährleistet und eine nahtlose Integration ermöglicht werden.

Hinweise zum Einsatz von KI

Geben Sie an, welche KI-Werkzeuge Sie eingesetzt haben und welche Code-Abschnitte explizit mit KI-Werkzeugen erstellt wurden. Die Nutzung von KI setzt voraus, dass Sie den Code verstehen und dessen Funktionsweise und Interaktion mit der restlichen Applikation erklären können. Wenn Sie KI zum Ausarbeiten der Projektaufgabe einsetzen, dann darf das maximal in einem Frage-Modus erfolgen. Es ist keinesfalls erlaubt KI in einem Agentenmodus zu betreiben. In diesem Fall wäre Ihre Eigenleistung negativ zu beurteilen.

Hinweise zur Präsentation

Am Semesterende findet sowohl für SWK als auch für WEA eine getrennte Präsentation Ihres Projekts statt.

Ablauf SWK-Präsentation

1. Live-Demonstration des gesamten API-relevanten Funktionsumfangs auf Request-Ebene (z. B. in Postman, Visual Studio Code etc.) – unter Berücksichtigung möglicher Änderungswünsche durch die Vortragenden (andere Speisekarte, andere Bestellung etc.)
2. Detailliertere Besprechung von zwei Fragen (eine darf von Ihnen nominiert werden, eine vom Vortragenden)

Ablauf WEA-Präsentation

Live-Demonstration der gesamten Funktionalität in einem logischen Ablauf. Die Präsentation ist eine Einzelpräsentation, die anderen Studierenden müssen nicht die ganze Zeit über anwesend sein.

Hinweise zur Bewertung

Folgende Kriterien werden in die Beurteilung der Projektarbeit einbezogen:

- ☐ Qualität des Quelltexts sowie der gewählten Architektur
- ☐ Qualität Frontend (weniger Designskills, mehr durchdachtes und benutzerfreundliches User-Interface)
- ☐ Qualität Dokumentation (Beantwortung der Fragen)
- ☐ Gesamteindruck Präsentation (strukturierte, gut vorbereitete Demo)
- ☐ Vollständigkeit der Implementierung
- ☐ Korrektheit der Implementierung

- ☐ Verbale Erläuterung der Implementierung
- ☐ Termintreue